

4. Логістична регресія та SVM

У цьому розділі ми переходимо від прогнозування чисел до класифікації об'єктів. Розглядаються два популярні алгоритми класифікації: логістична регресія (Logistic Regression) та метод опорних векторів (Support Vector Machine)

Перед початком роботи з алгоритмами класифікації необхідно підключити бібліотеки,

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
from sklearn.preprocessing import StandardScaler
```

які відповідають за обчислення, підготовку даних, побудову моделей класифікації та оцінку їх якості:

- **make_classification** - генератор штучних даних для задач класифікації. Дозволяє швидко створити набір ознак та міток класів;
- **LogisticRegression** - реалізація алгоритму логістичної регресії для бінарної та мультикласової класифікації;
- **SVC (Support Vector Classifier)** - реалізація методу опорних векторів (SVM), який шукає оптимальну межу між класами;
- **classification_report** - інструмент для виведення основних метрик класифікації: precision, recall, f1-score та accuracy;
- **confusion_matrix** - створює матрицю помилок класифікації;
- **ConfusionMatrixDisplay** - використовується для графічного відображення confusion matrix.

Суть класифікації та логістична регресія

У задачах класифікації модель повинна визначити, до якого класу належить об'єкт. Тобто, якщо регресія повертає дійсне число, то класифікація прогнозує категорію.

```
X, y = make_classification(n_samples=500, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Тут `make_classification` створює: 500 об'єктів (`n_samples=500`), кожен об'єкт містить дві ознаки (`n_features=2`), немає лінійно залежних ознак (`n_redundant=0`), кожен клас створює один кластер точок (`n_clusters_per_class=1`).

Основою логістичної регресії є сигмоїдна функція (sigmoid function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Функція приймає будь-яке число та повертає значення від 0 до 1. Наприклад 0.95 - модель майже впевнена у класі 1, 0.10 - ймовірніше клас 0.

Після обчислення ймовірності модель формує Decision Boundary - межу прийняття рішень, яка розділяє класи у просторі ознак. Для логістичної регресії вона зазвичай є прямою лінією:

$$w_1x_1 + w_2x_2 + b = 0$$

Модель намагається знайти таку межу, щоб максимально добре розділити об'єкти різних класів.

Метрики якості класифікації

Після підготовки даних та масштабування можна перейти до навчання моделі логістичної регресії.

```
log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train)

print("--- Logistic Regression Results ---")
y_pred_log = log_reg.predict(X_test_scaled)
print(classification_report(y_test, y_pred_log))
```

Загальна точність (Accuracy) показує частку правильних відповідей.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Але якщо дані не збалансовані (один клас зустрічається набагато частіше за інший), загальна точність може бути оманливою. Наприклад, якщо 95% листів - це звичайні повідомлення, а лише 5% - спам, модель може завжди відповідати «не спам» і все одно отримати accuracy 95%. Хоча насправді така модель нічому не навчилась.

Матриця помилок (Confusion Matrix) - таблиця помилок класифікації, містить: **TP (True Positive)** - правильне визначення позитивного класу; **TN (True Negative)** - правильне визначення негативного класу; **FP (False Positive)** - клас хибно визначено позитивним; **FN (False Negative)** - клас хибно визначено негативним.

Точність (Precision) - показує, скільки позитивних прогнозів були правильними.

$$Precision = \frac{TP}{TP + FP}$$

Повнота (Recall) - показує, скільки реальних позитивних об'єктів знайшла модель.

$$Recall = \frac{TP}{TP + FN}$$

Тобто, припустимо є 100 листів, 10 з них - спам. Модель позначила як спам 8 листів, але з цього її списку лише 5 виявилися дійсно спамом. Тоді:

$$Precision = \frac{5}{8}$$

$$Recall = \frac{5}{10}$$

Наскільки можна довіряти позитивним прогнозам моделі?

Яку частину всього спаму модель змогла знайти?

F1-score поєднує precision та recall:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Ця метрика особливо корисна для незбалансованих даних.

Метод опорних векторів (SVM)

SVM (Support Vector Machine) шукає гіперплощину, яка максимально добре розділяє класи. Головна мета - не просто повернути межу, а зробити максимальний зовнішній відступ (margin) між класами.

```
svm_model = SVC(kernel='linear', C=1.0, probability=True)
svm_model.fit(X_train_scaled, y_train)

print("--- SVM Results ---")
y_pred_svm = svm_model.predict(X_test_scaled)
print(classification_report(y_test, y_pred_svm))
```

SVC() створює модель методу опорних векторів. Тут: `kernel='linear'` - використовується лінійна межа розділення; `C=1.0` - параметр регуляризації; `probability=True` - дозволяє моделі оцінювати ймовірність класів.

Під час навчання SVM намагається знайти оптимальну межу між класами з максимальним margin. Margin (зовнішній відступ) - це відстань між decision boundary та найближчими точками.

$$Margin = \frac{2}{\|w\|^1}$$

Чим більший margin, тим стабільнішою зазвичай є модель на нових даних.

Після навчання використовується:

```
y_pred_svm = svm_model.predict(X_test_scaled)
```

Метод predict() - прогнозує класи для тестової вибірки.

¹ $\|w\|$ - довжина (норма) вектора.

Опорні вектори (support vectors) – це точки, які знаходяться найближче до межі. Саме вони визначають положення decision boundary. Якщо видалити інші точки, то межа майже не зміниться.

Параметр C контролює баланс між шириною margin та кількістю помилок класифікації. Чим більше C – тим менше помилок допускає модель, але більший ризик перенавчання.

Якщо дані не можна розділити прямою лінією, використовується kernel trick. Kernel переводить дані у простір вищої розмірності, де вони вже можуть бути лінійно роздільними.

У SVM доступні різні типи kernel: **linear** – лінійна межа; **rbf** – нелінійна межа; **poly** – поліноміальна межа.

Візуалізація Decision Boundary

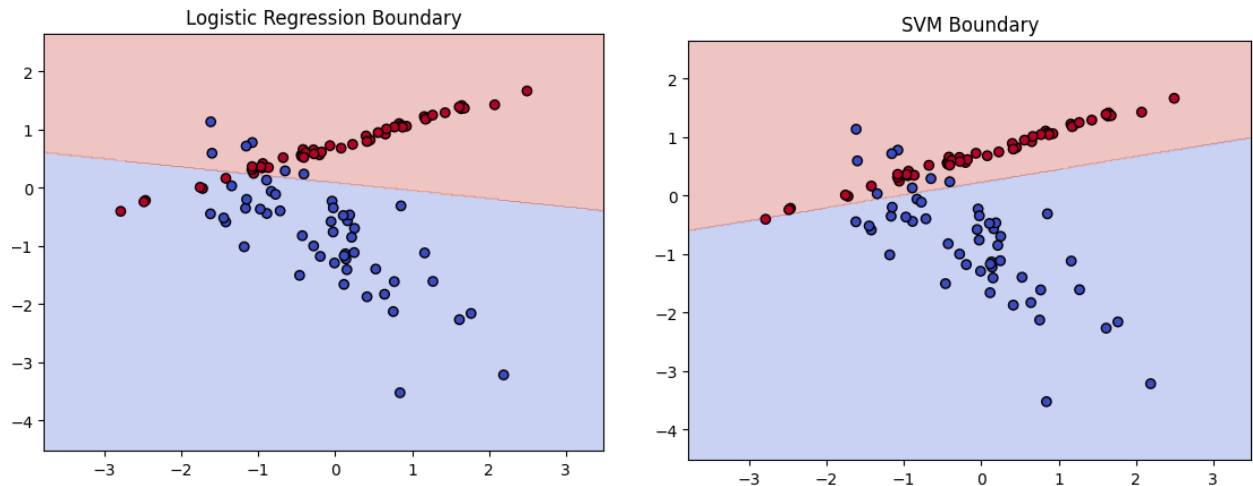
Візуальне порівняння логістичної регресії та SVM:

```
def plot_decision_boundary(model, X, y, title):
    h = .02
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min,
y_max, h))

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k',
cmap=plt.cm.coolwarm)
    plt.title(title)
    plt.show()

plot_decision_boundary(log_reg, X_test_scaled, y_test, "Logistic
Regression Boundary")
plot_decision_boundary(svm_model, X_test_scaled, y_test, "SVM Boundary")
```



Функція `plot_decision_boundary()` створює сітку точок у просторі ознак та визначає, до якого класу модель відносить кожну точку.

Для цього: `meshgrid()` - створює координатну сітку; `predict()` - класифікує всі точки сітки; `contourf()` - зафарбовує області різних класів; `scatter()` - відображає реальні точки даних.

Матриця помилок

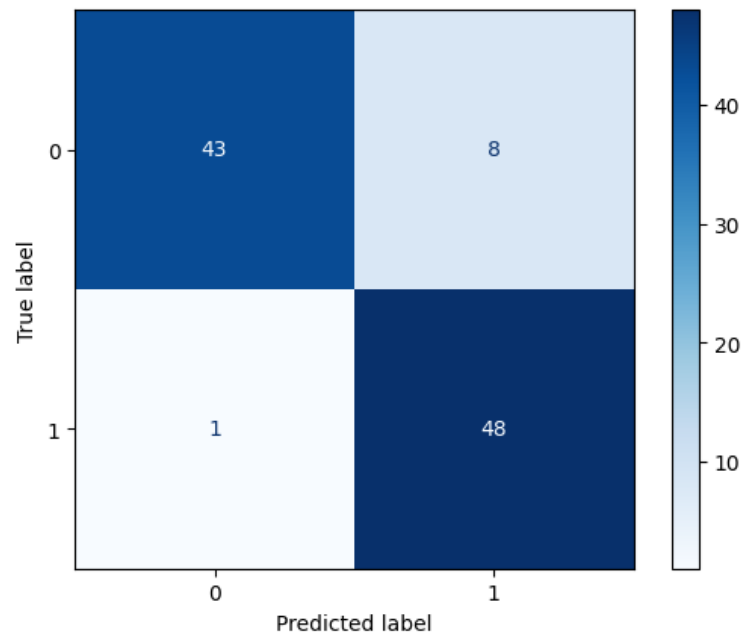
Для детального аналізу роботи класифікатора використовується confusion matrix - матриця помилок класифікації.

```
print("--- Візуалізація помилок (Confusion Matrix) ---")
ConfusionMatrixDisplay.from_estimator(svm_model, X_test_scaled, y_test,
                                     cmap='Blues')
plt.show()
```

`ConfusionMatrixDisplay.from_estimator()` автоматично виконує прогнозування, обчислює матрицю помилок та будує її графічне відображення.

Тут: `svm_model` - навчена модель класифікації; `X_test_scaled` - тестові ознаки; `y_test` - правильні мітки класів; `cmap='Blues'` - кольорова схема візуалізації.

Після цього `plt.show()` відображає графік на екрані.



Основні кольори для матриці помилок

Відтінкові (одноколірні) схеми			
Blues	синій	Reds	червоний
Greens	зелений	Purples	фіолетовий
Oranges	помаранчевий	Greys	сірий
Перехідні (градієнтні) схеми			
viridis	фіолетовий - зелений	plasma	фіолетовий - жовтий
inferno	чорний - червоний - жовтий	Magma	темний - рожево- фіолетовий
cividis	жовто-зелений		

Ресурси

Файл з наведеними лістингами: [Лекція 4. Логістична регресія та SVM.pdf](#)